# CS8.401: Principles of Information Security

Programming Assignments Specification

## Contents

**Introduction: The Cryptographic Clique** **2**

**1 Part 0: Minicrypt Clique Web Explorer** **3**

**2 Part I: Symmetric Cryptography & Minicrypt** **4**

**3 Part II: Hashing and Data Integrity** **5**

**4 Part III: Cryptomania (Public-Key Cryptography)** **6**

**5 Part IV: Secure Multi-Party Computation** **7**

# Introduction: The Cryptographic Clique

A central theme of these assignments is the **Universality of Cryptographic Primitives**. In modern cryptography, we want to minimize assumptions. If we can prove that a One-Way Function (OWF) exists, we can theoretically build Pseudorandom Generators (PRGs), Pseudorandom Functions (PRFs), Pseudorandom Permutations (PRPs), Collision-Resistant Hash Functions (CRHFs), HMAC, Secure MACs, and Digital Signatures.

This forms a tightly knit *clique* of reductions. Every primitive in the clique implies every other — and you are required to implement **both directions** of each reduction (construction and reverse reduction). In practice, we instantiate these with concrete primitives: **AES** (a PRP/PRF) or the **Discrete Logarithm Problem** (a OWF/OWP). By writing your code modularly, you should be able to plug either foundation into any primitive in the chain.

**Global Rules (apply to all assignments)**

- **No external cryptographic libraries.** Every primitive used must be your own prior implementation. You may use OS-level randomness (`os.urandom`) and big-integer arithmetic only.
- **Bidirectional reductions.** For every clique pair $(A, B)$ you must implement both $A \Rightarrow B$ (construction) and $B \Rightarrow A$ (reduction). Partial credit for one direction only.
- **Interactive demo.** Each assignment includes a live browser demo (React, toy parameters) that makes the construction visible. Graders will interact with the demo, not just read test output.
- **Lineage.** Later assignments must call your own earlier implementations. PA#18 must use your PA#12/#16 PKC; PA#20 must call your PA#18/#19.

# 1 Part 0: Minicrypt Clique Web Explorer

**Intuition: Making Reductions Visible**

The web app is a single React site that grows with you as you complete Parts I–IV. It has two columns: Column 1 builds the *source primitive* $A$ from a concrete foundation (AES or DLP); Column 2 applies the abstract reduction $A \to B$. Column 2 receives $A$ as a black box and must not touch the foundation directly.

### PA #0: Minicrypt Clique Web Explorer

- **PA #0:** Build a two-column React app. Top bar: foundation toggle (AES / DLP). Column 1: select source primitive $A$; shows the concrete construction chain from the foundation with live hex values at each step. Column 2: select target primitive $B$; shows the abstract reduction $A \to B$ step-by-step. Bottom panel: collapsible proof summary (theorem names and security claim per step). Unimplemented primitives show a stub with the target PA number. App must be live and grader-interactive from day one.

# 2 Part I: Symmetric Cryptography & Minicrypt

**Intuition: Bootstrapping Security from a OWF**

A true random pad (OTP) gives perfect security but requires keys as long as the message. By leveraging computational hardness, a short random seed feeds a PRG to produce an effectively infinite pseudorandom stream. Upgrading to a PRF gives random-access to this stream, enabling encryption and authentication. Everything here is equivalent: OWF $\Leftrightarrow$ PRG $\Leftrightarrow$ PRF $\Leftrightarrow$ PRP $\Leftrightarrow$ MAC. Implement each with both reduction directions.

### PA #1 & #2: The Foundation

- **PA #1:** Implement a One-Way Function (OWF) — choose DLP ($f(x) = g^x \bmod p$) or AES-based ($f(k) = \mathrm{AES}_k(0^{128}) \oplus k$). Construct a secure Pseudorandom Generator (PRG) from it using the HILL hard-core-bit construction. Also implement the reverse: show PRG $\Rightarrow$ OWF. Include a NIST SP 800-22 statistical test suite (frequency, runs, serial tests).
- **PA #2:** Use the PRG $G$ from PA #1 to construct a PRF $F_k()$ via the **GGM tree construction**: $F_k(b_1 \cdots b_n) = G_{b_n}(\cdots G_{b_1}(k))$. Also implement the reverse: PRF $\Rightarrow$ PRG via $G(s) = F_s(0) \| F_s(1)$. Support AES as a drop-in substitute for $F_k$.

### PA #3 & #4: Confidentiality

- **PA #3:** Use the PRF from PA #2 to build a CPA-secure encryption scheme. Standard construction: $C = \langle r,\ F_k(r) \oplus m \rangle$. Demonstrate the broken deterministic variant (nonce reuse).
- **PA #4:** Use a PRF/PRP (from PA #2/#3, or your own AES) to implement three modes of operation:
  1. **CBC** (Cipher Block Chaining)
  2. **OFB** (Output Feedback)
  3. **Randomized CTR** (Counter Mode)

  Include attack demos: CBC IV-reuse and OFB keystream-reuse.

### PA #5 & #6: Integrity and Active Security

- **PA #5:** Design secure MACs from your PRF (PA #2). Implement: (1) PRF-MAC $t = F_k(m)$; (2) CBC-MAC for variable-length messages. Show MAC $\Rightarrow$ PRF (backward direction). Demonstrate EUF-CMA security. Stub out HMAC (to be completed in PA #10).
- **PA #6:** Combine the MAC (PA #5) and CPA-secure encryption (PA #3) to build a provably **CCA-secure** encryption scheme via **Encrypt-then-MAC**. Verify MAC *before* decrypting; return $\perp$ on failure. Demonstrate the malleability attack on CPA-only encryption and show it is blocked by the MAC.

# 3 Part II: Hashing and Data Integrity

**Intuition: Domain Extension**

We need hash functions to compress arbitrary-length inputs into fixed-length digests. The **Merkle-Damgård transform** extends a fixed-length compression function to arbitrary lengths. The **Birthday Bound** ($O(2^{n/2})$ for $n$-bit output) is the absolute security ceiling for any hash function. **HMAC** bridges hashing back into Minicrypt: CRHF $\Leftrightarrow$ HMAC $\Leftrightarrow$ MAC, extending the clique.

## PA #7 & #8: Provable Hashing

- **PA #7:** Implement the **Merkle-Damgård Transform** as a generic framework accepting any compression function $h$ as a parameter. Include MD-strengthening padding ($M\|1\|0^*\|\langle|M|\rangle$). Show collision propagation: a collision in $h$ becomes a collision in $H$.
- **PA #8:** Use the MD transform (PA #7) with the DLP-based compression function $h(x, y) = g^x \cdot \hat{h}^y \bmod p$ to produce a provably collision-resistant hash function. Security proof: finding a collision implies solving DLP. Run the birthday attack on a truncated 16-bit version to confirm the $O(2^{n/2})$ bound empirically.

## PA #9: Birthday Attack

- **PA #9:** Implement the birthday attack in two variants: (1) naive sort-based ($O(k \log k)$ time, $O(k)$ space) and (2) Floyd's cycle detection ($O(2^{n/2})$ time, $O(1)$ space). Attack your PA #8 hash with $n \in \{8, 10, 12, 14, 16\}$-bit output; plot empirical collision counts against the theoretical curve $1 - e^{-k^2/2^n}$.

## PA #10: HMAC and HMAC-Based CCA Encryption

- **PA #10:** Implement **HMAC** over your PA #8 hash: $\mathrm{HMAC}_k(m) = H\big((k \oplus opad)\|H((k \oplus ipad)\|m)\big)$. Show both directions: CRHF $\Rightarrow$ MAC (HMAC is a secure EUF-CMA MAC) and MAC $\Rightarrow$ CRHF (use HMAC as a MD compression function). Demonstrate the **length-extension attack** on naive $H(k\|m)$ and show HMAC defeats it. Implement **Encrypt-then-HMAC** as a second CCA-secure scheme alongside PA #6. Use constant-time tag comparison.

# 4 Part III: Cryptomania (Public-Key Cryptography)

**Intuition: The Key Distribution Problem**

Symmetric cryptography requires a pre-shared key. PKC solves this using trapdoor permutations and hard problems (Factoring, DLP). The jump from Minicrypt to Cryptomania requires a *new* computational assumption beyond OWF existence.

### PA #11, #12, #13, #14: Mathematical Foundations

- **PA #11:** Implement **Diffie-Hellman Key Exchange**. Both parties establish a shared secret $K = g^{ab}$ over a public channel. Demonstrate the MITM attack (unauthenticated DH is broken). Use your PA #13 to generate safe primes.
- **PA #12:** Implement **Textbook RSA** and **PKCS#1 v1.5** padded RSA. Use PA #13 for prime generation; implement extended Euclidean and square-and-multiply yourself. Demonstrate: (1) determinism attack on textbook RSA; (2) Bleichenbacher padding oracle on a toy 512-bit modulus; (3) expose $(p, q, d_p, d_q, q_{inv})$ for PA #14.
- **PA #13:** Implement **Miller-Rabin Primality Testing**. Show Carmichael number $n = 561$ passes Fermat but is caught by Miller-Rabin. Generate primes and benchmark against the Prime Number Theorem ($O(\ln n)$ candidates expected).
- **PA #14:** Implement the **Chinese Remainder Theorem** (CRT) and apply it two ways: (1) **CRT-based RSA decryption** (Garner's algorithm, $\approx 4\times$ speedup); (2) **Håstad's broadcast attack** — given $e = 3$ ciphertexts $c_i = m^3 \bmod N_i$ to three different recipients, recover $m$ via CRT + integer cube root (no private key needed). Show that PKCS#1 v1.5 padding defeats the attack.

### PA #15, #16, #17: Secure PKC Systems

- **PA #15:** Implement **Digital Signatures** (RSA hash-then-sign: $\sigma = H(m)^d \bmod N$). Show existential forgery on raw (unhashed) RSA signatures via multiplicative homomorphism. Implement EUF-CMA security game.
- **PA #16:** Implement the **ElGamal PKC** (based on DDH hardness). Demonstrate malleability: $(c_1, 2c_2 \bmod p)$ decrypts to $2m$, showing ElGamal is only CPA-secure.
- **PA #17:** Combine Digital Signatures (PA #15) and ElGamal (PA #16) to build **CCA-secure PKC** via **Encrypt-then-Sign**. Verify signature *before* decrypting. Show the malleability attack from PA #16 is now blocked ($\perp$ returned). Full lineage: PA #17 $\rightarrow$ PA #15+#16 $\rightarrow$ PA #12+#11 $\rightarrow$ PA #13.

# 5 Part IV: Secure Multi-Party Computation

**Intuition: Computing on Encrypted Data**

How can two mutually distrusting parties compute a joint function without revealing their inputs? The foundation of all MPC is **Oblivious Transfer (OT)**. OT $\Rightarrow$ Secure AND. Since any function is a boolean circuit of AND and XOR gates, Secure AND + Secure XOR $\Rightarrow$ any 2-party computation securely.

### PA #18, #19, #20: The Pinnacle of Cryptography

- **PA #18:** Implement **1-out-of-2 Oblivious Transfer** using your PKC from PA #12 or #16 (Bellare-Micali protocol). Receiver gets $m_b$ and learns nothing about $m_{1-b}$; sender learns nothing about $b$. Expose a clean 3-step API for PA #19.
- **PA #19:** Use OT (PA #18) to implement **Secure AND**: Alice inputs $a$, Bob inputs $b$, both learn $a \wedge b$, neither learns the other's bit. Also implement **Secure XOR** (free via additive secret sharing, no OT needed) and **NOT** (local flip, free).
- **PA #20:** Use Secure AND (PA #19) and Secure XOR to securely evaluate **any** 2-party boolean function (Yao's Garbled Circuits / GMW). Implement a boolean circuit evaluator (DAG of AND/XOR/NOT gates) and three mandatory test circuits: (1) **Millionaire's Problem** ($x > y$ without revealing $x$ or $y$); (2) **Secure Equality** ($x = y$); (3) **Secure Addition** ($x + y \bmod 2^n$). Report OT call count and wall-clock time for $n = 8$ bits.